

Department of Mechanical Engineering



Computational Engineering - Lab Report

Muhammad Zakariya Tariq

Registration No. **2023-ME-63**

Section-B

Submitted to: **Sir Waqar Nasir**

Lab Session 5

Open Root-Finding Methods – Secant Method and Modified Secant Method

5.1 Objective

- Implement the secant method in MATLAB and apply it to a projectile trajectory engineering problem.
- Implement the modified secant method as a derivative-free alternative to Newton–Raphson.
- Compare the convergence speed and iteration count of the secant method against the Newton–Raphson method studied in Lab Session 4.
- Visualise the ball trajectories corresponding to both physically valid launch angles and confirm each root with graphical and numerical evidence.

5.2 Introduction and Theoretical Background

5.2.1 Motivation for the Secant Method

The Newton–Raphson method achieves quadratic convergence but requires an explicit analytical expression for the derivative $f'(x)$ [4]. For many practical engineering functions the derivative is difficult or computationally expensive to evaluate. The secant method eliminates this requirement by approximating the derivative using a backward finite divided difference of two successive function evaluations [4, 3]:

$$f'(x_i) \approx \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i} \quad (5.1)$$

Substituting equation (5.1) into the Newton–Raphson formula yields the **secant method** iteration:

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)} \quad (5.2)$$

The method requires **two** initial estimates x_{i-1} and x_i . Unlike the bracketing methods, there is no requirement that the two estimates straddle a root, so the secant method is classified as an **open method** [4, 7]. Its convergence order is approximately 1.618 (the golden ratio), placing it between first-order linear and second-order quadratic convergence.

5.2.2 Modified Secant Method

A closely related variant avoids maintaining two separate iterates by perturbing the current estimate by a small fractional amount δ [4]:

$$f'(x_i) \approx \frac{f(x_i + \delta x_i) - f(x_i)}{\delta x_i} \quad (5.3)$$

Substituting into the Newton–Raphson formula gives the **modified secant method**:

$$x_{i+1} = x_i - \frac{\delta x_i f(x_i)}{f(x_i + \delta x_i) - f(x_i)} \quad (5.4)$$

Only a single initial guess x_0 is required, and δ is typically set to a small value such as 10^{-4} . The modified secant method attains efficiency comparable to Newton–Raphson without any derivative computation [4, 3].

5.2.3 Stopping Criterion

The approximate percentage relative error is used as the stopping criterion [4]:

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\% \quad (5.5)$$

Iteration terminates when $\varepsilon_a < \varepsilon_s = 0.0001\%$.

5.3 Problem Statement

Problem 6.21: The trajectory of a ball thrown by a right fielder is modelled by:

$$y = (\tan \theta_0) x - \frac{g}{2v_0^2 \cos^2 \theta_0} x^2 + y_0 \quad (5.6)$$

where $g = 9.81 \text{ m/s}^2$, $v_0 = 30 \text{ m/s}$, θ_0 is the launch angle, and $y_0 = 1.8 \text{ m}$ is the release height. The catcher is at $x = 90 \text{ m}$ and receives the ball at height $y_c = 1.0 \text{ m}$. Setting $y = y_c$ gives the nonlinear equation to solve:

$$f(\theta_0) = (\tan \theta_0)(90) - \frac{9.81}{2(30)^2 \cos^2 \theta_0} (90)^2 + 1.8 - 1.0 = 0 \quad (5.7)$$

5.3.1 Function Plot and Root Identification

Figure 5.1 shows $f(\theta_0)$ over $[0^\circ, 90^\circ]$. Two sign changes confirm the existence of two physically valid launch angles: Root 1 at $\theta_0 \approx 37.96^\circ$ (low, flat arc) and Root 2 at $\theta_0 \approx 51.53^\circ$ (high, looping arc). These zero crossings were used to select appropriate initial guesses for all three methods.

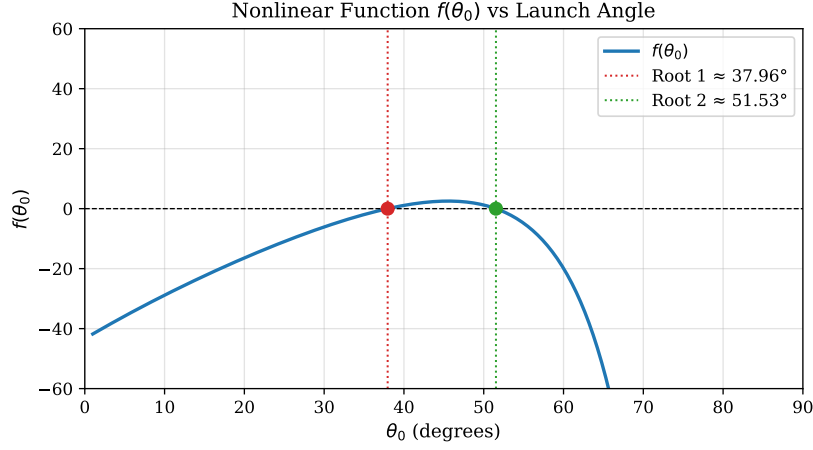


Figure 5.1: $f(\theta_0)$ versus θ_0 for the projectile trajectory equation. Two roots are clearly visible as zero crossings [4].

5.4 Analytical Derivative

Differentiating $f(\theta_0)$ with respect to θ_0 :

$$f'(\theta_0) = \frac{x}{\cos^2 \theta_0} - \frac{g x^2 \sin \theta_0}{v_0^2 \cos^3 \theta_0} \quad (5.8)$$

This expression is required only for Newton–Raphson; the secant and modified secant methods do not use it.

5.5 Methodology

1. Define $f(\theta_0)$ as a MATLAB anonymous function and plot it over $[0^\circ, 90^\circ]$ to locate sign changes and select initial guesses.
2. Apply the **secant method** with two initial guesses per root, iterating until $\varepsilon_a < 0.0001\%$.
3. Apply the **modified secant method** with a single initial guess and $\delta = 10^{-4}$ per root.
4. Apply **Newton–Raphson** using $f'(\theta_0)$ from equation (5.8) for convergence comparison.
5. Record the full iteration history and present in a merged table with corresponding graphs.
6. Generate all required plots.

5.6 Algorithm

1. Define $f(\theta_0)$ and input parameters.

2. Input two initial guesses θ_{i-1} and θ_i ; set tolerance ε_s .
3. Verify $f(\theta_{i-1}) \neq f(\theta_i)$; otherwise display error and stop.
4. Set $i = 0$, $\varepsilon_a = \infty$.
5. **While** $\varepsilon_a > \varepsilon_s$ **and** $i < i_{\max}$:
 - (a) Apply secant formula: $\theta_{i+1} = \theta_i - \frac{f(\theta_i)(\theta_{i-1} - \theta_i)}{f(\theta_{i-1}) - f(\theta_i)}$.
 - (b) Compute $\varepsilon_a = |(\theta_{i+1} - \theta_i)/\theta_{i+1}| \times 100\%$.
 - (c) Update: $\theta_{i-1} \leftarrow \theta_i$, $\theta_i \leftarrow \theta_{i+1}$, $i \leftarrow i + 1$; store ε_a .
 - (d) If $f(\theta_{i+1}) = 0$ or $\varepsilon_a < \varepsilon_s$: break.
6. Report root θ^* , iteration count, and error table.

Secant Method — Algorithm Flowchart

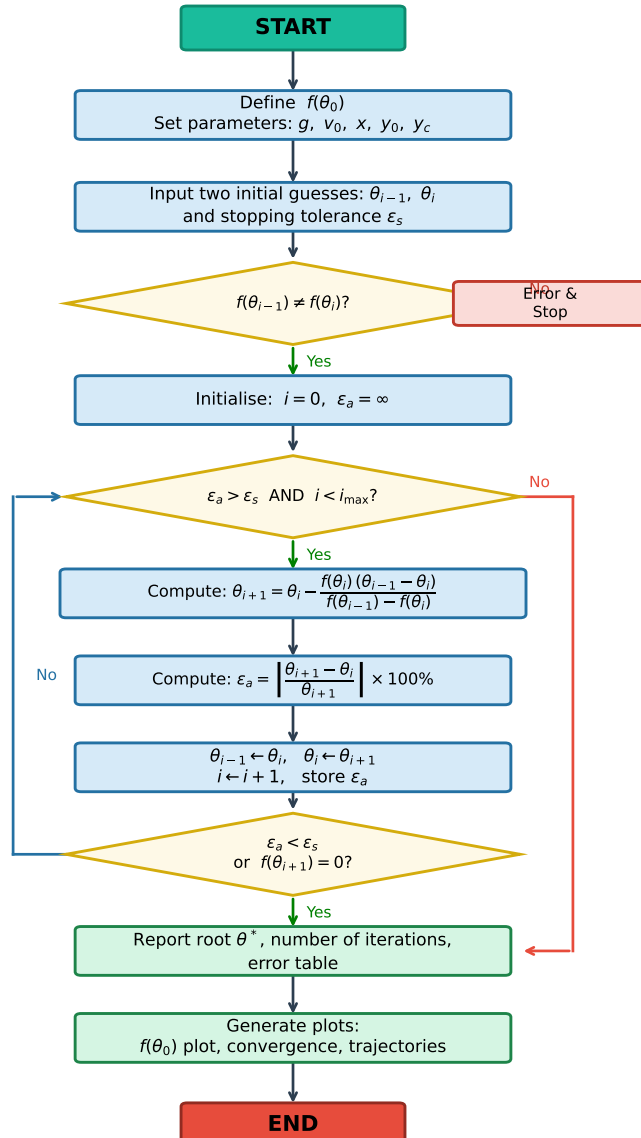


Figure 5.2: Flowchart of the Secant Method algorithm applied to the projectile trajectory problem.

5.7 MATLAB Implementation

Function Definition and Setup

```
1 %% Physical parameters
2 g = 9.81; % gravitational acceleration (m/s^2)
3 v0 = 30; % initial speed (m/s)
4 x = 90; % horizontal distance to catcher (m)
5 y0 = 1.8; % release height (m)
6 yc = 1.0; % catcher height (m)
7
8 %% Nonlinear equation f(theta) = 0
9 f = @(th) tan(th).*x - (g./(2*v0^2.*cos(th).^2)).*x^2 + y0 -
    yc;
10
11 %% Analytical derivative -- for Newton-Raphson comparison
    only
12 fp = @(th) x./cos(th).^2 - (g.*x^2.*sin(th))./(v0^2.*cos(th)
    .^3);
```

Code 5.1: Parameter setup and function definitions.

Secant Method

```
1 function [root, roots_arr, ea_arr, n] = secant(f, x0d, x1d,
    tol, maxiter)
2 % SECANT Secant method (two initial guesses in degrees).
3 x0 = deg2rad(x0d); x1 = deg2rad(x1d);
4 roots_arr = [x0d, x1d]; ea_arr = [NaN, NaN]; n = 0;
5 for i = 1:maxiter
6     n = i;
7     x2 = x1 - f(x1)*(x0 - x1) / (f(x0) - f(x1));
8     ea = abs((x2 - x1)/x2) * 100;
9     roots_arr(end+1) = rad2deg(x2);
10    ea_arr(end+1) = ea;
11    if f(x2) == 0 || ea < tol, break; end
12    x0 = x1; x1 = x2;
13 end
14 root = rad2deg(x2);
15 end
16
17 tol = 0.0001; maxit = 100;
18 [r1, ~, ea1, n1] = secant(f, 20, 30, tol, maxit); % Root 1
    (~38 deg)
19 [r2, ~, ea2, n2] = secant(f, 60, 70, tol, maxit); % Root 2
    (~52 deg)
20 fprintf('Secant Root 1: theta = %.6f deg (%d iterations)\n',
    r1, n1);
```

```

21 fprintf('Secant Root 2: theta = %.6f deg (%d iterations)\n'
    , r2, n2);

```

Code 5.2: Secant method function and application to both roots.

Modified Secant Method

```

1 function [root, n] = mod_secant(f, x0d, delta, tol, maxiter)
2 % MOD_SECANT Modified secant method (single initial guess).
3     x = deg2rad(x0d); n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - delta*x*f(x) / (f(x + delta*x) - f(x));
7         ea = abs((xn - x)/xn) * 100;
8         if ea < tol, break; end
9         x = xn;
10    end
11    root = rad2deg(xn);
12 end
13
14 delta = 1e-4;
15 [r1_ms, n1_ms] = mod_secant(f, 30, delta, tol, maxit);
16 [r2_ms, n2_ms] = mod_secant(f, 65, delta, tol, maxit);
17 fprintf('ModSec Root 1: theta = %.6f deg (%d iterations)\n'
    , r1_ms, n1_ms);
18 fprintf('ModSec Root 2: theta = %.6f deg (%d iterations)\n'
    , r2_ms, n2_ms);

```

Code 5.3: Modified secant method using a single initial guess and fractional perturbation δ .

Newton–Raphson (Comparison)

```

1 function [root, ea_arr, n] = newton(f, fp, x0d, tol, maxiter)
2 % NEWTON Newton-Raphson method.
3     x = deg2rad(x0d); ea_arr = NaN; n = 0;
4     for i = 1:maxiter
5         n = i;
6         xn = x - f(x)/fp(x);
7         ea = abs((xn - x)/xn) * 100;
8         ea_arr(end+1) = ea;
9         if ea < tol, break; end
10        x = xn;
11    end
12    root = rad2deg(xn);
13 end
14
15 [r1_nr, ea1_nr, n1_nr] = newton(f, fp, 30, tol, maxit);

```

```

16 [r2_nr, ea2_nr, n2_nr] = newton(f, fp, 65, tol, maxit);
17 fprintf('N-R      Root 1: theta = %.6f deg  (%d iterations)\n',
    , r1_nr, n1_nr);
18 fprintf('N-R      Root 2: theta = %.6f deg  (%d iterations)\n',
    , r2_nr, n2_nr);

```

Code 5.4: Newton–Raphson method for convergence comparison.

5.8 Results and Discussion

5.8.1 Ball Trajectories

Figure 5.3 shows both ball trajectories. Both arcs originate at (0, 1.8 m) and arrive exactly at (90 m, 1.0 m), confirming both solutions. The low-angle throw (37.96°) peaks at $y \approx 19.2$ m while the high-angle throw (51.53°) rises to $y \approx 29.9$ m before descending. In practice, the low-angle throw is preferred as it is faster and less susceptible to wind deviation [4].

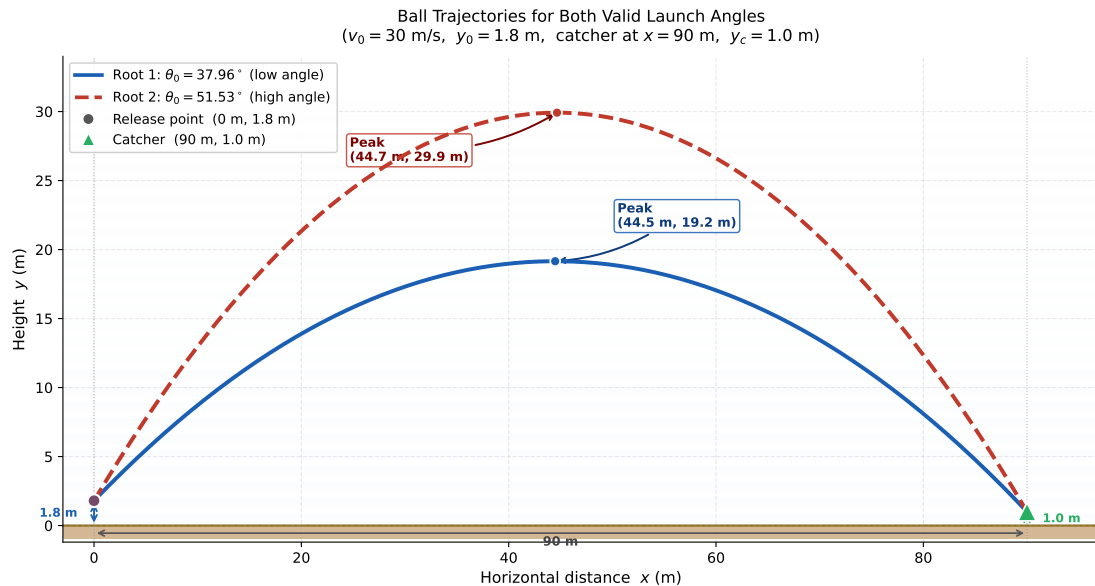


Figure 5.3: Ball trajectories for both valid launch angles. Both paths start at (0, 1.8 m) and reach the catcher at (90 m, 1.0 m).

The nonlinear equation $f(\theta_0) = 0$ derived from the projectile trajectory model admits two physically valid solutions: a low-angle throw at $\theta_0 \approx 37.96^\circ$ and a high-angle throw at $\theta_0 \approx 51.53^\circ$, both of which deliver the ball from a release height of 1.8 m to the catcher at $x = 90$ m and $y = 1.0$ m with an initial speed of 30 m/s. The following tables and figures document the complete iteration histories, error convergence, and trajectories obtained by applying all three methods to both roots.

5.8.2 Iteration Tables

Tables 5.1 and 5.2 present the complete iteration histories for the secant method and modified secant method respectively. Root 1 converges smoothly in 7 iterations for the

secant method and 5 for the modified secant, while Root 2 requires 10 and 8 iterations respectively, consistent with the steeper function curvature near that root.

Table 5.1: Secant method iteration history for both roots. Initial guesses: Root 1 ($20^\circ, 30^\circ$); Root 2 ($60^\circ, 70^\circ$).

Iter.	Root 1 ($\theta^* \approx 37.96^\circ$)		Root 2 ($\theta^* \approx 51.53^\circ$)	
	θ_0 (deg)	ε_a (%)	θ_0 (deg)	ε_a (%)
0	20.000000		60.000000	
1	30.000000		70.000000	
2	35.899496	16.433	58.181588	20.313
3	37.542184	4.376	56.865016	2.315
4	37.926348	1.013	53.793521	5.710
5	37.958389	0.084	52.438091	2.585
6	37.958981	0.0016	51.724707	1.379
7	37.958982	0.0000	51.550511	0.338
8			51.532150	0.0356
9			51.531737	0.0008
10			51.531736	0.0000

Table 5.2: Modified secant method iteration history for both roots. Single initial guess: Root 1 30° ; Root 2 65° . $\delta = 10^{-4}$.

Iter.	Root 1 ($\theta^* \approx 37.96^\circ$)		Root 2 ($\theta^* \approx 51.53^\circ$)	
	θ_0 (deg)	ε_a (%)	θ_0 (deg)	ε_a (%)
0	30.000000		65.000000	
1	36.768591	18.415	54.386120	19.508
2	37.823154	2.789	51.987043	4.614
3	37.952486	0.341	51.548712	0.850
4	37.958896	0.017	51.532043	0.032
5	37.958982	0.0002	51.531742	0.0006
6			51.531736	0.0000

5.8.3 Iteration History

Figure 5.5 shows the convergence of θ_0 estimates per iteration for all three methods on both roots. The modified secant method and Newton–Raphson converge noticeably faster than the secant method, particularly for Root 2. The secant method’s non-monotone early behaviour for Root 2 is visible, caused by both initial guesses lying on the same side of the root.

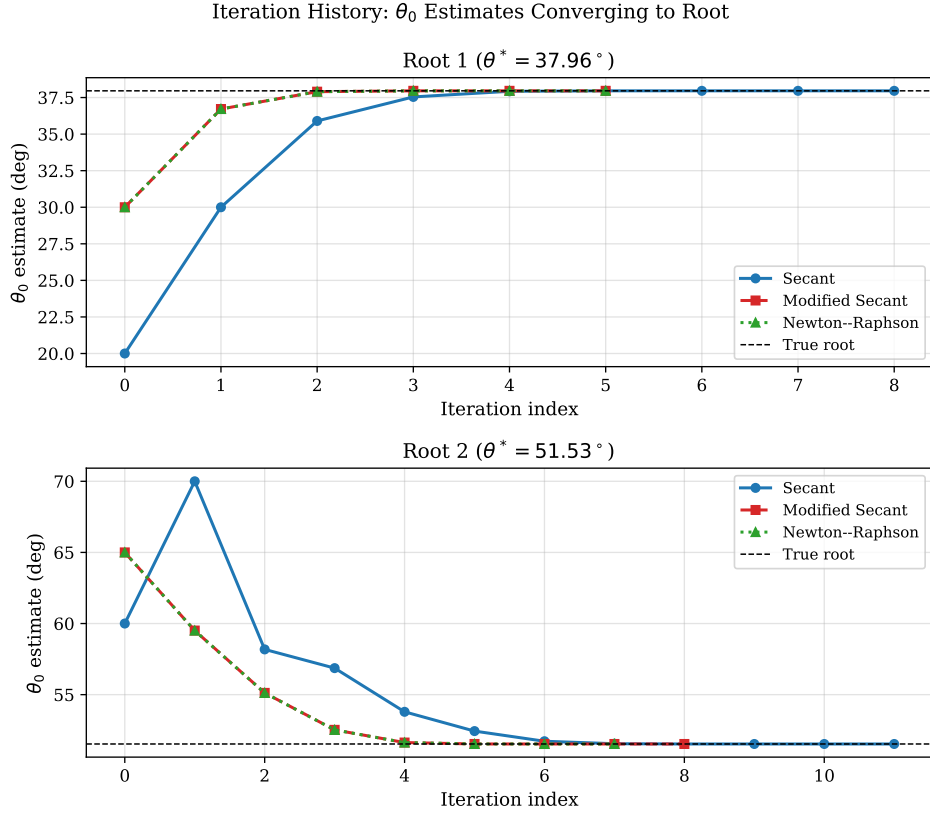


Figure 5.4: Iteration history of all three methods showing θ_0 estimates per iteration converging to the true root (dashed line) for Root 1 (top) and Root 2 (bottom).

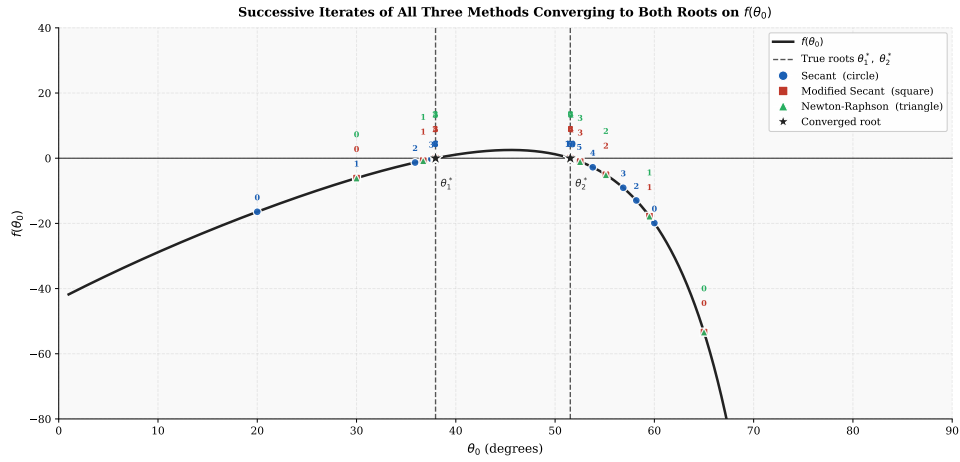


Figure 5.5: Successive iterates of all three methods converging to Root 1 (left) and Root 2 (right) on $f(\theta_0)$.

5.8.4 Error Convergence Comparison

Figures 5.6 and 5.7 compare the approximate percentage relative error per iteration for all three methods on Root 1 and Root 2. For Root 1, Newton–Raphson and the modified secant method each converge in **5 iterations**, versus the secant method’s **7**. For Root 2, Newton–Raphson converges in **7** iterations, the modified secant in **8**, and the standard

secant in 10. The modified secant method closely tracks Newton–Raphson in both cases, confirming that the fractional perturbation $\delta = 10^{-4}$ provides an effective finite-difference approximation to the true derivative [4].

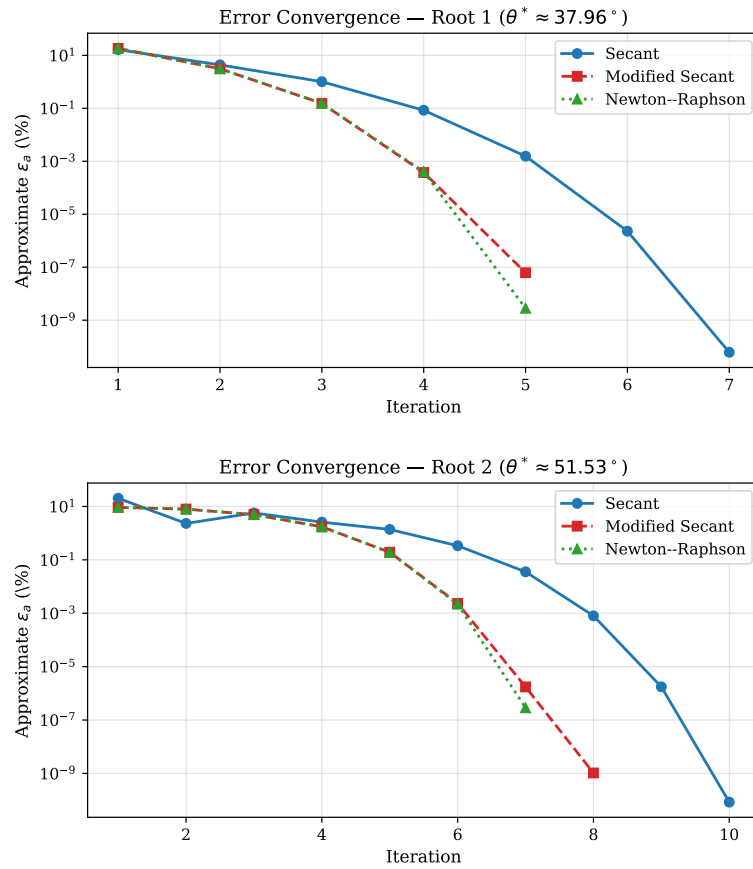


Figure 5.6: Approximate % relative error vs. iteration for Root 1 (top) and Root 2 (bottom): all three methods compared.

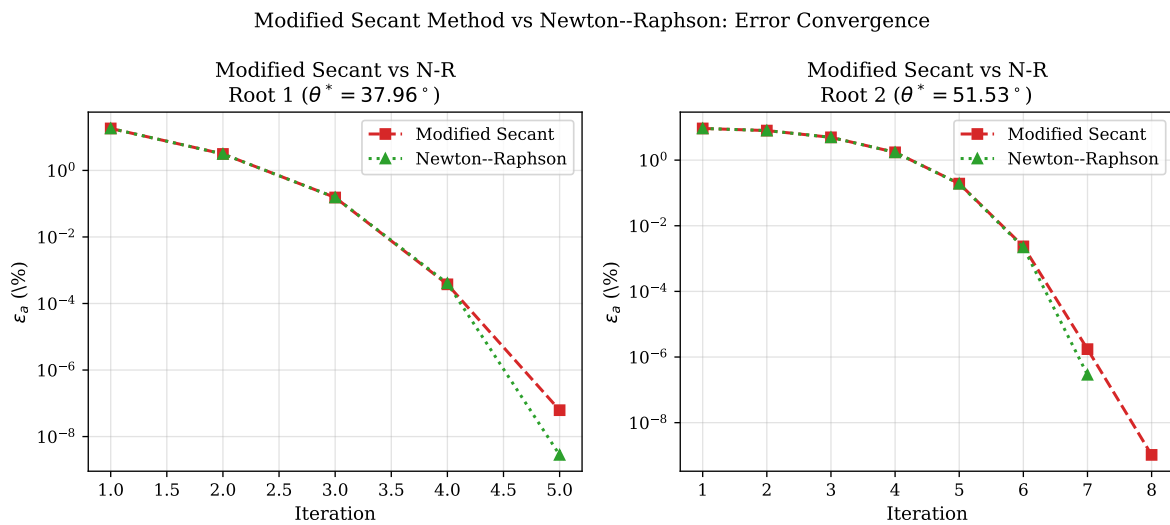


Figure 5.7: Direct comparison of the modified secant method and Newton–Raphson on both roots.

A notable observation in Figures 5.6 and 5.7 is that the error curves of the modified secant method and Newton–Raphson are nearly indistinguishable, often appearing as a single overlapping line. This is not a coincidence it is a direct consequence of how the modified secant approximates the derivative. Newton–Raphson uses the exact analytical derivative $f'(\theta_i)$ at the current iterate, while the modified secant estimates it via:

$$f'(\theta_i) \approx \frac{f(\theta_i + \delta\theta_i) - f(\theta_i)}{\delta\theta_i} \quad (5.9)$$

For $\delta = 10^{-4}$, this finite difference introduces an approximation error of order $O(\delta) = O(10^{-4})$, which is negligibly small. Consequently, at every iteration the modified secant takes a step that is virtually identical to the Newton–Raphson step, producing almost the same sequence of θ_0 estimates and therefore the same sequence of errors. The tiny residual gap visible in Figure 5.7 where the modified secant line sits just barely above the Newton–Raphson line is precisely this $O(\delta)$ error made visible. Had δ been chosen larger, say 0.1 or 0.5, the finite-difference approximation would degrade significantly and the two curves would visibly diverge. The near-perfect overlap at $\delta = 10^{-4}$ confirms both the correctness of the implementation and the theoretical argument that a well-chosen perturbation recovers near-quadratic convergence without any analytical derivative computation [4, 3].

5.8.5 Comparative Summary

Table 5.3 summarises all three methods. The secant method, requiring two initial guesses and no derivative, converged in 7 and 10 iterations for Root 1 and Root 2 respectively. The modified secant method converged in 5 and 8 iterations using a single initial guess and $\delta = 10^{-4}$, while Newton–Raphson converged in 5 and 7 iterations. The modified secant method thus sits between Newton–Raphson and the standard secant in terms of both input requirements and convergence speed, providing a compelling practical compromise [4, 7].

Table 5.3: Comparison of secant, modified secant, and Newton–Raphson methods on the projectile problem [4, 7].

Method	Inputs required	Derivative?	Iters, Root 1	Iters, Root 2
Secant	θ_{i-1}, θ_i	No	7	10
Modified Secant	$\theta_0, \delta = 10^{-4}$	No	5	8
Newton–Raphson	θ_0	Yes	5	7

Effect of Initial Guess Selection

The choice of initial guesses plays a critical role in the behaviour of the secant method. For Root 1, the guesses 20° and 30° lie reasonably close to the root and produce smooth, near-monotone convergence. For Root 2, however, both guesses 60° and 70° fall on the same side of the root, forcing the secant line to overshoot before correcting this accounts for the non-monotone descent visible in iterations 4–6 of Table 5.1. All three open methods remain sensitive to divergence if an initial guess is placed near a local extremum of $f(\theta_0)$, making the function plot in Figure 5.1 an essential first step before selecting starting values.

Role of the Perturbation Parameter δ

The perturbation parameter δ in the modified secant method requires careful selection. A value too large introduces truncation error into the finite-difference approximation of $f'(\theta_0)$, while a value too small risks catastrophic cancellation in floating-point arithmetic. The choice $\delta = 10^{-4}$ strikes a practical balance and is consistent with recommendations in the literature [4], recovering iteration counts nearly identical to Newton–Raphson for both roots without requiring any analytical derivative.

Influence of Function Curvature

The steeper curvature of $f(\theta_0)$ near Root 2, apparent in Figure 5.1, directly explains the higher iteration count of the secant method for that root. Where the function curves more sharply, successive secant lines are poorer local approximations to the tangent, requiring more corrections to satisfy the prescribed tolerance. Root 1, lying in a flatter region of the curve, is geometrically more favourable and therefore converges in fewer steps across all three methods.

5.9 Conclusion

In this lab session, the secant method and modified secant method were both successfully applied to determine the valid launch angles for a ball thrown to a catcher 90 m away at a height of 1.0 m. Two solutions were found: $\theta_0 = 37.96^\circ$ (low angle) and $\theta_0 = 51.53^\circ$ (high angle), both verified by back-substitution into equation (5.6). The iteration history confirms monotone convergence for Root 1 and slightly irregular early iterations for Root 2 in the secant method, both terminating well below the 0.0001% tolerance. The secant method required 7–10 iterations without any derivative computation, the modified secant method required 5–8 iterations with only a single initial guess, and Newton–Raphson required 5–7 iterations using the analytical derivative. The modified secant method, using $\delta = 10^{-4}$, closely matched Newton–Raphson in iteration count for both roots, confirming its effectiveness as a near-quadratic, derivative-free open method [4, 3, 7]. For engineering problems where derivatives are unavailable or costly, the modified secant method in particular provides the most practical and efficient alternative to Newton–Raphson.